1 Peripherals and Buses of the MPS2-AN385 Platform

Before writing a driver, it is necessary to know the hardware it talks to: which board is emulated, which peripherals it contains, where they sit in the address space, and how the CPU reaches them.

1.1 CMSDK Hardware and CMSIS Software Conventions

Two Arm standards cover the two sides of a peripheral:

- **CMSDK** defines the hardware blocks,
- **CMSIS** defines how software names and accesses them.

Aspect	CMSDK	CMSIS
Full name	Cortex-M System Design Kit	Cortex Microcontroller Software Interface Standard
Nature	**hardware**: ready-made peripheral designs placed next to a Cortex-M core	**software**: a standard for the C headers and helper functions that describe a chip
Examples	APB UART, APB timer and dual timer, watchdog, AHB GPIO	`core_cm3.h`, `NVIC_EnableIRQ()`, `UART0->DATA`, `UART0TX_IRQn`
Key property	registers documented once by Arm, identical on every chip that integrates the block	shipped by chip vendors with their parts
In this context	the peripherals of the MPS2-AN385 board	a register header written by hand **in CMSIS style**, without vendor libraries, so names match real vendor code

1.2 Board, Clock, and Peripheral Address Map

The code runs on an emulated Arm development board. This section identifies the board, the single clock that times everything, and the addresses where its peripherals are located.

Element	Description
MPS2	Arm development board whose main chip is an **FPGA**: a reprogrammable chip whose circuit is loaded at power-on, so the same board can imitate different microcontrollers
AN385	the circuit (FPGA image, described in Arm Application Note 385) used here: a **Cortex-M3** CPU plus CMSDK peripherals
QEMU `mps2-an385`	emulator of exactly this system: the code runs on the PC without the physical board
Clock	one **25 MHz** clock shared by CPU and peripherals; every timing value (UART speed, timer period) is computed from it
Peripheral region	addresses `0x40000000` – `0x5FFFFFFF`, reserved by the Cortex-M memory map for devices; each device occupies a 4 KiB window

Figure 4.1 — MPS2-AN385 peripheral region (0x4000_0000…)

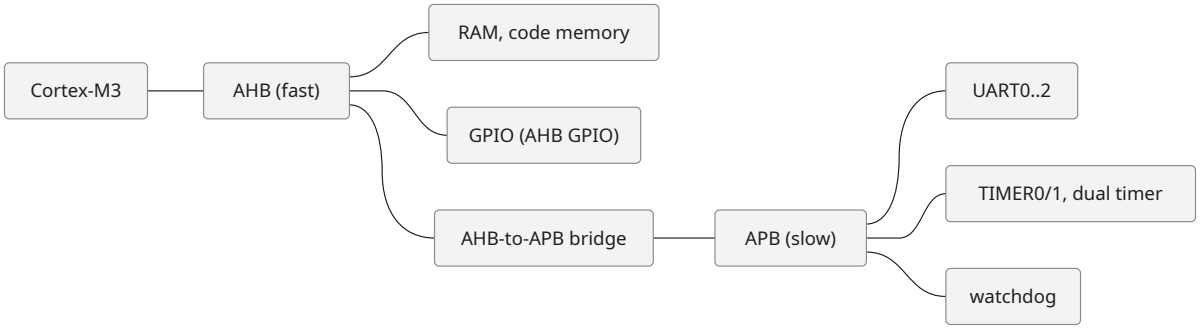


1.3 AHB and APB Buses

The CPU does not talk to every device in the same way: fast and slow devices sit on two different buses, connected by a bridge.

A **bus** carries every load and store from the CPU to a memory or a device.

Figure 4.2 — AHB and APB buses of the MPS2-AN385



Bus	Characteristics
AHB (Advanced High-performance Bus)	pipelined, one transfer per cycle: memories, GPIO
APB (Advanced Peripheral Bus)	simple, slow, low-power: rarely accessed registers (UART, timers, watchdog)
AHB-to-APB bridge	translates each access: a few extra cycles per APB register

For software **the bus is transparent**: only the timing differs. Software does not need to know which bus a device is on: every register access is the same load or store instruction, and the bridge routes it automatically. The only difference is speed: an APB access takes a few extra clock cycles.

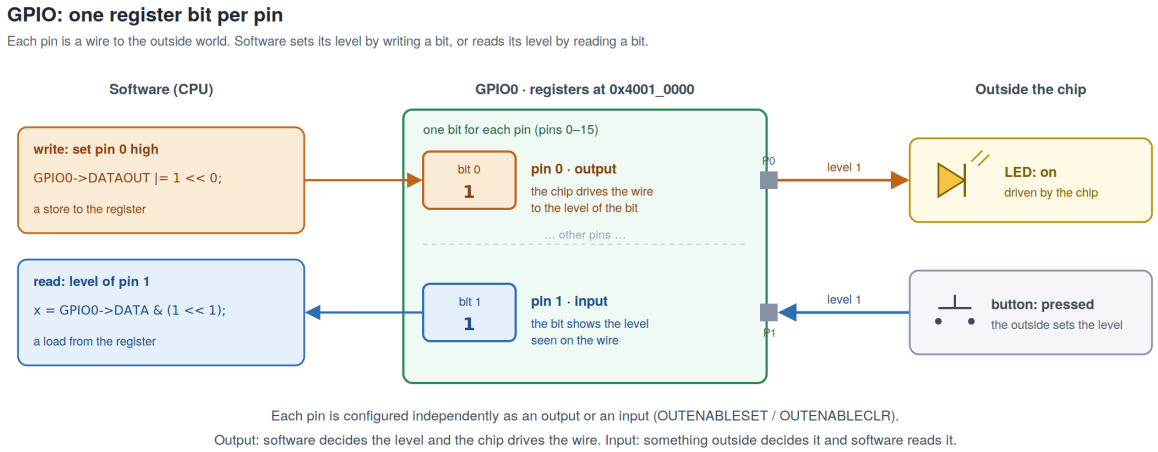
1.4 Standard I/O Interfaces and DMA

Microcontrollers exchange data with the outside world through a small set of standard **interfaces**; a DMA controller can additionally move data without using the CPU.

Interface	Wires	Operating principle	Typical use
GPIO	1 per pin	each pin is a digital input (level read) or output (driven to 0 or 1)	buttons, LEDs, chip-selects
UART	2 (TX, RX)	asynchronous: no clock wire, both sides agree on a **baud rate**	debug console, PC link
SPI	clock, data out, data in, 1 select per device	synchronous; the controller drives the clock, one bit out and one in per tick; **chip-select** picks the device	flash, displays (MHz)
I2C	2 (clock, data)	synchronous multi-drop; the controller first sends the 7-bit device **address**	sensors (100–400 kHz)
ADC / DAC	1 per channel	analog voltage to number (ADC) or number to voltage (DAC)	temperature, audio

1.4.1 GPIO

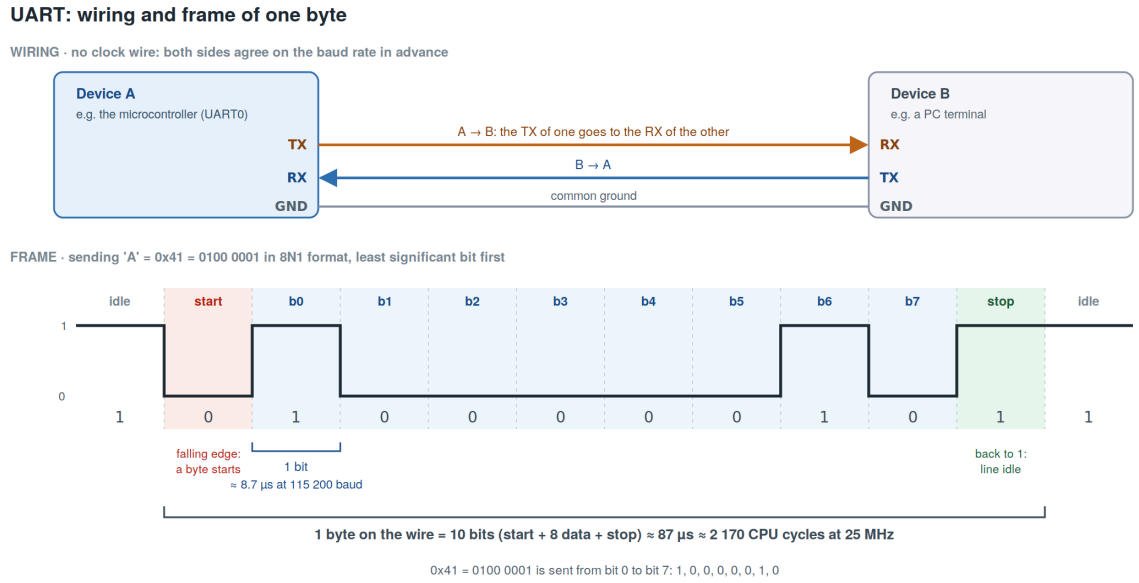
A **GPIO** (General-Purpose Input/Output) pin is the simplest interface: a **single wire** whose level the software can **read** (input) or **set** to 0 or 1 (output). When a pin is used as an output, the electrical circuit that drives it determines whether the pin can safely share its wire with other devices.



1.4.2 UART

A **UART** (Universal Asynchronous Receiver-Transmitter) sends bytes one bit at a time over a single wire per direction. It is **asynchronous**: there is no clock wire, so transmitter and receiver must agree in advance on the speed, the **baud rate** (bits per second), and each side times the bits with its own clock.

Element	Description
Wires	TX (transmit) and RX (receive), crossed between the two devices: the TX of one goes to the RX of the other; plus a common ground
Idle line	held at 1 when nothing is sent
Start bit	one 0 bit: the falling edge tells the receiver that a byte is starting and synchronizes its timing
Data bits	usually 8, sent **least significant bit first**
Parity bit	optional error check (none in the common **8N1** format: 8 data bits, no parity, 1 stop bit)
Stop bit	one 1 bit: returns the line to idle before the next byte



Transmission of `'A'` (`0x41` = `0100 0001`) in 8N1 format, bit 0 first:

```
idle  start  b0  b1  b2  b3  b4  b5  b6  b7  stop  idle
 1      0     1   0   0   0   0   0   1   0    1     1
```

Each byte therefore takes **10 bits** on the wire (start + 8 data + stop). At 115 200 baud one bit lasts about 8.7 µs, so one byte takes about **87 µs**: a long time for a CPU running at 25 MHz, which is why the way the CPU waits for the UART (polling or interrupts) matters.

1.4.3 DMA

Moving data between a peripheral and memory normally costs CPU time: the CPU must read each byte from the device and store it in memory, or the opposite. A **DMA controller** (Direct Memory Access) is a separate hardware unit that **performs these copies** on its own, **leaving the CPU free** to execute other code while the transfer is in progress.

Aspect	Description
Operation	copies source → destination for a count, using the bus while the CPU runs; usually signals completion with an interrupt
Directions	peripheral → memory (UART RX, ADC); memory → peripheral (SPI TX)
Costs	bus contention; the buffer must not be touched during the transfer
AN385	no DMA: the CPU moves every byte

2 Memory-Mapped I/O

On a Cortex-M, devices have **no special I/O instructions**: each device register is a **memory address**, and the CPU controls the device with ordinary loads and stores.

With **memory-mapped I/O** (MMIO), every peripheral register has a fixed physical address and is accessed with

ordinary load and store instructions:

```
#define UART0_DATA (*(volatile uint32_t *)0x40004000u)

UART0_DATA = 'A';      /* a store to 0x40004000: the UART transmits 'A' */
```

The assignment compiles to a single `str` at `0x40004000`: the syntax is a variable access, the meaning is defined by the hardware.

2.1 Device Registers Are Not RAM

Although a device register is accessed like a variable, **it does not behave like memory:** reading or writing it can trigger actions in the hardware.

Property of a device register	Example
A **read** can have a side effect	consuming a received byte, acknowledging an event
A **write** can start an action	a transmission, a flag clear, a pin change
The value can **change without any store**	a status bit set when a byte arrives
The value read back can **differ** from the value written	read-only bits, self-clearing bits, reserved bits that must be written as zero or preserved

2.2 The `volatile` Qualifier

The C compiler assumes that **memory changes only when the program writes it**. For device registers this assumption is false, and `volatile` tells the compiler not to rely on it.

Under optimization the compiler caches values, removes repeated reads, and merges stores: all invalid for device registers.

> ⚠ **WHAT `VOLATILE` DOES AND DOES NOT DO**
>
> It forces every source access to be performed, in order with respect to other `volatile` accesses. It gives **no atomicity** and **no synchronization** between `main` and a handler.

2.3 Read-Modify-Write Hazards

A plain assignment to a register, `REG = value;`, is a single store: one instruction, which cannot be interrupted halfway. A compound assignment such as `|=`, `&=` (or operation like `x++`) looks like one operation too, but the CPU performs it in three steps:

```
CMSDK_GPIO0->DATAOUT |= (1u << 0);      /* set bit 0 of the output latch */
```

```
    ldr  r1, [r0]       /* 1. READ   the whole register into r1   */
    orr  r1, r1, #1     /* 2. MODIFY bit 0 inside the CPU         */
```

```
    str  r1, [r0]       /* 3. WRITE  the whole value back        */
```

Between the read and the write, the **register can change**, or the value written back can do more than intended.

2.3.1 Two Kinds of Register

To change one pin without touching the others, the GPIO (`0x40010000`) offers two kinds of register:

- **registers that hold all the pins together**, such as `DATAOUT` (the levels driven on the output pins): a store overwrites every pin, so changing one pin requires a read-modify-write;
- **set/clear pairs**, such as `OUTENABLESET` and `OUTENABLECLR` (make pins outputs or inputs): a store acts only on the bits written as 1 and ignores the bits written as 0, so a single plain store changes one pin and no read is needed.

```
CMSDK_GPIO0->OUTENABLESET = (1u << 0);     /* safe: one store, pin 0 becomes an output */
CMSDK_GPIO0->DATAOUT     |= (1u << 0);     /* risky: read-modify-write of all the pins */
```

2.3.2 When Read-Modify-Write Fails

Hazard	What goes wrong	Example
An interrupt in between	another context changes the register after the read; the write puts back the old value and cancels that change	`main` reads `DATAOUT` to set bit 0; an interrupt sets bit 1; `main` writes back the value it read, and bit 1 is lost
Write-1-to-clear bits	in some registers, writing a 1 *clears* a flag; writing back the value read clears every flag that was set	UART `INTSTATUS` reads `0b11` (TX and RX pending); `INTSTATUS
The read is not what was written	the value read describes something else; writing it back changes bits that were never meant to be touched	GPIO `DATA` reads the levels of all pins, inputs included; writing it back drives the outputs with the levels just observed

2.3.3 Safe Alternatives

- **Use a set/clear register** whenever the device has one (GPIO `OUTENABLESET` / `OUTENABLECLR` , NVIC `ISER` / `ICER`): no read at all.
- **Write only the bit, with a plain store**, in write-1-to-clear registers: `CMSDK_UART0->INTSTATUS = UART_INT_TX;` clears TX and leaves the other flags alone.
- **Protect the read-modify-write** when it cannot be avoided: mask interrupts around it (`irq_save()` … `irq_restore()` , see the section on critical sections), or do it before interrupts are enabled, as the boot code does for `CCR` .

2.4 Register Overlays with C Structures

Instead of one macro per register, a whole register block can be described by a C `struct` placed at the device's base address, so that every register becomes a named field.

```c
typedef struct
{
    volatile uint32_t DATA;      /* 0x00 read = received byte, write = byte to send */
    volatile uint32_t STATE;     /* 0x04 status flags */
    volatile uint32_t CTRL;      /* 0x08 control flags */
    volatile uint32_t INTSTATUS; /* 0x0C interrupt status; write 1 to clear a bit */
    volatile uint32_t BAUDDIV;   /* 0x10 baud-rate divider */
} CMSDK_UART_TypeDef;

#define CMSDK_UART0   ( ( CMSDK_UART_TypeDef * ) 0x40004000UL )
```

Element	Meaning
Layout	fields at consecutive 4-byte offsets from the base address
`CMSDK_UART0`	a constant pointer, not a variable: `->STATE` is a load from `0x40004004`
Safety	`volatile` fields; bit names next to the `struct` (`UART_STATE_TXBF` is `1UL << 0`); `_Static_assert` + `offsetof` catch layout typos at compile time

The same technique describes SysTick, NVIC, SCB, timers, watchdog, GPIO, and FPGAIO.

2.5 Registers as a Hardware Protocol

Device registers must be used in a precise order, like the steps of a protocol; the **driver is the code that implements this order**.

Figure 4.3 — Register protocol of a device



Rule	Example
Configuration precedes enabling	speed before switching the transmitter on
A status check precedes every transfer	write `DATA` only when there is room, read it only when a byte is present
An event is acknowledged	clearing its flag lets the device report the next one

2.6 Peripherals under QEMU

QEMU **does not emulate every device of the board equally well**. At each address of the peripheral region there can be:

- **a real model**: QEMU imitates the device, and its registers react as on the hardware. Writing a byte to the UART `DATA` register prints it on the terminal, status bits change, interrupts arrive. A driver for such a device can be tested in the emulator. Examples: UART0, timers, SysTick, NVIC.

- **a placeholder**: QEMU only reserves the address, so that accesses do not fault, but there is no logic behind it. A driver for such a device cannot be tested, because right and wrong code behave the same. Example: GPIO0.
  - Writes are ignored
  - reads return 0
- **nothing**: no device answers at that address. Example: `0x40080000`.

The QEMU monitor shows which case applies at each address:

```
printf 'info mtree -f\nquit\n' | qemu-system-arm -M mps2-an385 -display none -S -monitor stdio
-serial null | grep ' 00000000400' | sort -u
```

```
0000000040000000-0000000040000fff (prio 0, i/o): cmsdk-apb-timer
0000000040002000-0000000040002fff (prio 0, i/o): cmsdk-apb-dualtimer
0000000040004000-0000000040004fff (prio 0, i/o): uart
0000000040008000-0000000040008fff (prio 0, i/o): cmsdk-apb-watchdog
0000000040010000-0000000040010fff (prio -1000, i/o): cmsdk-ahb-gpio
0000000040020000-0000000040020fff (prio 0, i/o): pl022
0000000040022000-0000000040022fff (prio 0, i/o): arm_sbcon_i2c
0000000040028000-0000000040028fff (prio 0, i/o): mps2-fpgaio
```

A real model appears with `prio 0` and its name, a placeholder with `prio -1000` (here the GPIO), and an empty address does not appear at all.

Running QEMU with `-d unimp` prints a line for every access to a placeholder or to an empty address. If it prints nothing, every device the program uses is a real model, so any malfunction is in the code.

> **CORE VERSUS BOARD PERIPHERALS**
>
> **Core** peripherals (SysTick, NVIC) have addresses fixed by ARMv7-M and exist on every Cortex-M. **Board** peripherals (UART0, GPIO0, FPGAIO) exist where the board places them, modelled as faithfully as the emulator allows.

3 Device Drivers and Polled UART I/O

The first driver built on these concepts is the UART, the serial port used as console. On top of it, a small formatting library provides `kprintf` for kernel output.

3.1 CMSDK UART Register Interface

The UART *(Universal Asynchronous Receiver-Transmitter)* sends and receives bytes on a serial line. It is controlled through five registers at `0x40004000`.

Offset	Register	Access	Content
`0x00`	`DATA`	R/W	write: byte to send; read: received byte (clears RXBF)

Offset	Register	Access	Content
`0x04`	`STATE`	R; bits 2–3 W1C	TXBF, RXBF, TXOR, RXOR
`0x08`	`CTRL`	R/W	TXEN, RXEN, TXIRQEN, RXIRQEN, overrun IRQ enables
`0x0C`	`INTSTATUS`	R/W1C	TX, RX, TXOR, RXOR interrupt pending
`0x10`	`BAUDDIV`	R/W	clock / baud rate, minimum 16

DATA 8-bit

UART DATA (0x00), low byte of the 32-bit register



Bits	Field	Description
7:0	DATA	Write: byte to send; read: received byte (clears RXBF)

STATE 8-bit

UART STATE (0x04), low byte of the 32-bit register



Bits	Field	Description
3	RXOR	Receive overrun: a byte was lost (write 1 to clear)
2	TXOR	Transmit overrun: a write was lost (write 1 to clear)
1	RXBF	Receive buffer full: a byte is waiting
0	TXBF	Transmit buffer full: the previous byte is still leaving

CTRL 8-bit = 0x03

UART CTRL (0x08) after uart_init, low byte



Bits	Field	Value	Description
5	RXORIRQEN	0	Receive-overrun interrupt enable
4	TXORIRQEN	0	Transmit-overrun interrupt enable
3	RXIRQEN	0	Receive interrupt enable
2	TXIRQEN	0	Transmit interrupt enable
1	RXEN	1	Receiver enable
0	TXEN	1	Transmitter enable

INTSTATUS 8-bit

UART INTSTATUS (0x0C), write 1 to clear a bit



Four facts about the UART shape the driver:

- **One byte of buffer per direction.** A byte written to `DATA` while the previous one is still leaving (`TXBF` = 1) is lost, and `TXOR` is set; a byte arriving while the previous one has not been read (`RXBF` = 1) is lost, and `RXOR` is set. The driver therefore checks `TXBF` before every write.
- **Off after reset.** `CTRL` is 0, so the transmitter and receiver are disabled and writes to `DATA` are silently ignored: `uart_init` must enable them first.
- **Speed on the real board.** `BAUDDIV` divides the 25 MHz clock: 25 000 000 / 115 200 ≈ 217 gives 115 200 baud.
- **Speed under QEMU.** The emulator ignores the line speed; any value from 16, the minimum, works, so the driver writes 16.

3.2 The Device Driver Abstraction

A driver separates *what* the rest of the system wants to do (send a byte) from *how* a particular device does it (which registers to touch, in which order). It is the only code that touches the device's registers; everything else calls its functions.

All character devices (UART, terminal, …) offer the same three operations. They can be collected in a `struct` whose fields are **pointers to functions**: each field holds the address of the function that performs that operation for one specific device.

```
struct char_driver
{
    void ( *init )( void );       /* bring the device up     */
    int  ( *read )( void );       /* one byte, or -1 if none */
    void ( *write )( char c );    /* send one byte           */
};

/* The UART driver fills the struct with its own functions */
static const struct char_driver uart0_driver = {
    uart_init,                    /* init  */
    uart_getc_polled,             /* read  */
    uart_putc_polled              /* write */
};

/* Generic code: works with any driver passed to it */
void say_hi( const struct char_driver *dev )
{
    dev->write( 'h' );            /* calls the write function of that driver */
    dev->write( 'i' );
}
```

`say_hi( &uart0_driver )` prints "hi" on the UART; passing another driver prints it on another device, without changing `say_hi`.

Aspect	Description
Interface	every driver offers the same operations (init, read, write); replacing the UART means replacing one driver file
Generalization	operating systems use this pattern for every class of device: Linux `struct file_operations` (`.open`, `.read`, `.write`, …) is filled in by each driver
This kernel	with a single console the `struct` is not needed: the kernel calls `uart_putc_polled` directly

3.3 Polled UART Driver

The simplest UART driver uses **polling**: the CPU repeatedly reads the status register until the device is ready, **without using interrupts**.

```
#include "drivers/uart.h"
#include "cmsdk.h"

void uart_init( void )
{
    CMSDK_UART0->BAUDDIV = 16;                          /* configure */
    CMSDK_UART0->CTRL = UART_CTRL_TXEN | UART_CTRL_RXEN;    /* enable */
}

void uart_putc_polled( char c )
{
    while ( CMSDK_UART0->STATE & UART_STATE_TXBF )          /* check status: room? */
    {
    }
    CMSDK_UART0->DATA = (uint32_t) (unsigned char) c;       /* move data */
}

int uart_getc_polled( void )
{
    if ( ( CMSDK_UART0->STATE & UART_STATE_RXBF ) == 0 )    /* check status: byte? */
    {
        return -1;
    }
    return (int) ( CMSDK_UART0->DATA & 0xFF );              /* reading DATA takes the byte */
}
```

Function	Protocol steps	Behaviour
`uart_init`	configure (`BAUDDIV`), enable (`CTRL`)	brings the UART to a known state
`uart_putc_polled`	check TXBF in a loop, write `DATA`	blocks until there is room

Function	Protocol steps	Behaviour
`uart_getc_polled`	check RXBF once, read `DATA`	never blocks: returns `-1` if no byte is waiting

Polled: status is checked by the CPU, not signalled by interrupts. The cast through `unsigned char` keeps a byte such as `0xE9` from becoming `0xFFFFFFE9` when `char` is signed.

> **ENABLE BEFORE MOVING DATA**
>
> Without `uart_init`, `CTRL` stays 0 and every byte is silently dropped (`-d guest_errors`: `transmit data write with Tx disabled`).

> **MOVE TO THE C APPENDIX**
>
> Section 3.4 (variadic functions) belongs in the appendix; leave a one-line reference in 3.5.

3.4 Variadic Functions and `<stdarg.h>`

A `printf`-like function accepts a variable number of arguments. C supports this with *variadic functions* and the macros of `<stdarg.h>`.

```
void kprintf( const char *fmt, ... )
{
    va_list ap;                     /* a cursor over the extra arguments    */
    va_start( ap, fmt );            /* start right after the last named one */
    int n = va_arg( ap, int );      /* take the next argument, as an int    */
    va_end( ap );                   /* done                                 */
}
```

Rule	Meaning
Argument count and types	unknown to the function: the **format string** supplies them; a mismatch is undefined behaviour
Default promotions	a `char` arrives as an `int`: `%c` uses `va_arg( ap, int )`
Freestanding	`<stdarg.h>` has only types and built-ins, no library code
Convention	a function taking a `va_list` is named `v…` (`vprintf`)

3.5 Building a Kernel printf over the UART

Without a C library there is no `printf`: the kernel builds its own, `kprintf`, which sends each character to the UART. The work is split in two:

- **formatting**: `vformat` turns the format string and its arguments into characters, one at a time;
- **output**: each character is passed to a function `out( c, ctx )`, which decides where it goes (for `kprintf`, the UART).

This way the same `vformat` can later serve other destinations.

```
static void put_string( void ( *out )( char c, void *ctx ), void *ctx, const char *s )
{
    while ( *s != '\0' )
    {
        out( *s, ctx );
        s++;
    }
}

void vformat( void ( *out )( char c, void *ctx ), void *ctx, const char *fmt, va_list ap )
{
    char num[12];   /* longest number + '\0' */

    while ( *fmt != '\0' )
    {
        if ( *fmt != '%' )              /* ordinary character: pass it on */
        {
            out( *fmt, ctx );
            fmt++;
            continue;
        }
        fmt++;                          /* a '%': look at the next letter */
        switch ( *fmt )
        {
        case 'c':
            out( (char) va_arg( ap, int ), ctx );   /* a char arrives as an int */
            break;
        case 's':
        {
            const char *s = va_arg( ap, const char * );
            put_string( out, ctx, s != 0 ? s : "(null)" );  /* no crash on null */
            break;
        }
        case 'd':
        {
            int v = va_arg( ap, int );
            uint32_t mag;
            if ( v < 0 ) { out( '-', ctx ); mag = 0u - (uint32_t) v; }
            else         { mag = (uint32_t) v; }
            utoa_dec( mag, num );
            put_string( out, ctx, num );
            break;
```

```
        }
        case 'u': utoa_dec( va_arg( ap, uint32_t ), num ); put_string( out, ctx, num ); break;
        case 'x': utoa_hex( va_arg( ap, uint32_t ), num ); put_string( out, ctx, num ); break;
        case '%': out( '%', ctx ); break;
        case '\0': out( '%', ctx ); return;          /* string ended right after '%' */
        default:  out( '%', ctx ); out( *fmt, ctx ); break;  /* unknown: as written */
        }
        fmt++;                          /* past the conversion letter */
    }
}
```

Aspect	Description
Conversions	`%c` , `%s` , `%d` , `%u` , `%x` (no `0x`), `%%` ; no width; unknown ones printed literally
`%d` magnitude	`0u - (uint32_t) v` , not `-v` : `-INT_MIN` overflows (UB), unsigned subtraction is always defined

```
static void kprintf_out( char c, void *ctx )
{
    (void) ctx;                         /* not used */
    uart_putc_polled( c );
}

void kprintf( const char *fmt, ... )
{
    va_list ap;
    va_start( ap, fmt );
    vformat( kprintf_out, 0, fmt, ap );
    va_end( ap );
}
```

`kprintf( "d=%d\n", -42 )` makes six `kprintf_out` calls. Polled, unbuffered, and interrupt-free, `kprintf` works in **any context**, fault handlers included.

3.6 Fault Handlers That Print the Cause

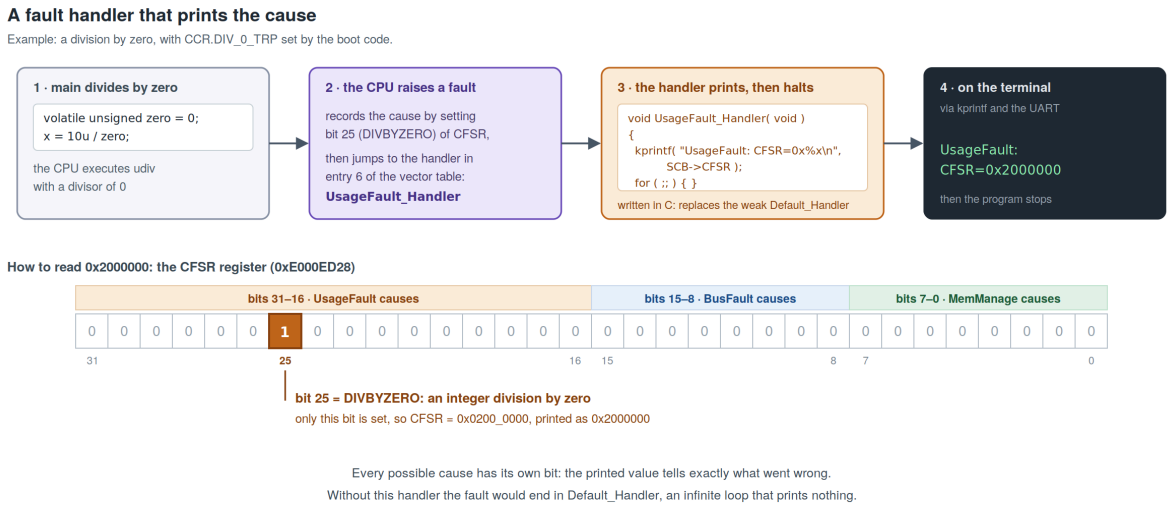
Without output, every fault ends in `Default_Handler` , an infinite loop that gives no clue about what went wrong. With `kprintf` , each fault can get its own handler, written in C, that replaces the weak default: it reads the fault status register of the core, prints it, and only then halts.

The CPU records the cause of a fault in one of two status registers, so each handler prints the one that matches its fault:

Register	Address	Content
CFSR (Configurable Fault Status)	`0xE000ED28`	one bit per cause of MemManage, BusFault, UsageFault

Register	Address	Content
HFSR (HardFault Status)	`0xE000ED2C`	why a HardFault occurred

> **DIVISION BY ZERO**
>
> With `CCR.DIV_0_TRP` set, an integer division by a `volatile` zero (`volatile` prevents compile-time evaluation) prints `UsageFault: CFSR=0x2000000` : bit 25, `DIVBYZERO` .



4 Polling and Interrupts

Polling wastes CPU time; interrupts let a device call the CPU only when needed. The hardware mechanism that enters and leaves a handler is described in the exception model of the chapter on the ARM Cortex-M3 architecture; this section compares the two approaches and shows what the mechanism means for driver code.

4.1 Polling versus Interrupts

There are two ways to find out when a device is ready: the CPU keeps asking (**polling**), or the device signals the CPU (**interrupt**). Each has its uses.

Aspect	Polling	Interrupts
CPU time	spent in the waiting loop even when nothing happens	spent only per event (entry, handler, exit)
Latency	up to one trip around the loop; grows with the devices checked	about 12 cycles of entry on Cortex-M3, nearly constant
Complexity	one loop	handler, shared data, priorities
CPU between events	busy	free, or asleep (`wfi`)
Suited to	certain, imminent events; contexts without interrupts (fault handlers)	rare or unpredictable events, long transfers

One byte at 115 200 baud lasts ~87 µs, about **2 170 cycles** at 25 MHz wasted by polling.

4.2 Interrupt Handlers Are Ordinary C Functions

On exception entry the Cortex-M3 pushes `r0` – `r3`, `r12`, `lr`, `PC`, and `xPSR` on the current stack and loads `EXC_RETURN` into `lr`. These are exactly the registers that the AAPCS allows a C function to clobber, and any compiled C function already preserves `r4` – `r11`. A device handler is therefore written as a plain C function whose name matches the vector table entry, and its ordinary return becomes an exception return. The register roles of the AAPCS, the stacked frame, `EXC_RETURN`, and tail-chaining are described in the Cortex-M3 chapter; the frame can be observed with gdb, as shown in the SysTick section below.

5 The SysTick Timer and Wraparound-Safe Time Keeping

An operating system needs a notion of time. SysTick, a **timer** built into every Cortex-M, generates a **periodic interrupt** (the *tick*) from which the kernel counts **milliseconds**.

5.1 SysTick Configuration

SysTick is the 24-bit down-counter built into every Cortex-M core. Its four registers (`CTRL`, `LOAD`, `VAL`, `CALIB` at `0xE000E010` – `0xE000E01C`), the choice of clock source, and the differences between SysTick and a device interrupt are described in the SysTick section of the chapter on the ARM Cortex-M3 architecture. The driver needs only three facts: one period lasts `LOAD + 1` cycles of the processor clock (25 MHz on `mps2-an385`); writing `CTRL = 7` starts the counter on the processor clock with the exception enabled; and SysTick is a core exception (number 15), so its priority is set in `SHPR3` and its handler needs no acknowledgement.

5.2 Reload Value and Tick Rate

The tick period is chosen by writing a reload value; it is computed from the clock frequency and the desired number of ticks per second.

From period = (`LOAD` + 1) / f: **`LOAD` = f / rate − 1**. At 25 MHz and 100 ticks/s, `LOAD` = 25 000 000 / 100 − 1 = **249 999** (`0x3D08F`), which fits in 24 bits; the longest period is 2^24 cycles ≈ 0.67 s.

Tick	Rate	Resolution	Exceptions/s	Note
1 ms	1 000 Hz	1 ms	1 000	in QEMU runs about 26% slow in wall-clock time
10 ms	100 Hz	10 ms	100	classic OS rate; exact wall-clock time in QEMU

A faster tick gives finer resolution at the cost of more exceptions.

5.3 SysTick Driver and Uptime Counter

The driver starts SysTick and counts elapsed milliseconds in a variable updated by the interrupt handler.

```
#define CPU_CLOCK_HZ 25000000UL     /* processor clock: 25 MHz */

static volatile uint32_t uptime_ms; /* changed by the handler */
```

```c
void systick_init( void )
{
    uptime_ms = 0;
    SysTick->LOAD = CPU_CLOCK_HZ / ( 1000 / TICK_MS ) - 1;   /* 249 999 for TICK_MS = 10 */
    SysTick->VAL = 0;               /* any write: counter = 0 */
    SysTick->CTRL = SysTick_CTRL_ENABLE | SysTick_CTRL_TICKINT | SysTick_CTRL_CLKSOURCE;
}

void SysTick_Handler( void )
{
    uptime_ms += TICK_MS;           /* no acknowledge needed */
}

uint32_t systick_uptime_ms( void )
{
    return uptime_ms;               /* one 32-bit read: never torn */
}
```

`SysTick_Handler` overrides the weak default in slot 15; `uptime_ms` is `volatile` because the handler changes it, and its single aligned 32-bit read can never be **torn**.

5.4 Wraparound-Safe Elapsed Time

The kernel measures time with a `uint32_t` counter of milliseconds (`uptime_ms`). A 32-bit variable can count only up to $2^{32}-1$ (`0xFFFFFFFF`); one millisecond later it **wraps around** to 0. This happens after 2^{32} ms ≈ **49.7 days**, a realistic uptime for an embedded device, so code that measures elapsed time must stay correct across the wrap.

> **ANALOGY: A 12-HOUR CLOCK**
>
> From 11 o'clock to 2 o'clock, 3 hours pass, even though 2 is smaller than 11. Counting on the clock face, $(2-11) \bmod 12 = 3$. Unsigned arithmetic in C works the same way, on a "clock" of 2^{32} positions: subtraction is computed **modulo** 2^{32}, so `now - start` is always the true distance from `start` to `now` , even if `now` has wrapped past 0.

Two ways to test whether `delay` ms have passed since `start` :

Test	Across the wrap	Why
`now - start >= delay`	**correct**	the subtraction gives the real elapsed time, modulo 2^32
`now >= start + delay`	**wrong**	`start + delay` can itself wrap to a small number, so the comparison becomes true too early

WORKED EXAMPLE: `START = 0XFFFFFF00`, `DELAY = 300` MS

`start` is 256 ms before the wrap (`0xFFFFFFFF` + 1 = 0).

Moment	`now`	`now - start` (correct test)	`start + delay` (wrong test)
16 ms after start	`0xFFFFFF10`	16 → 16 < 300: **not yet** ✔	wraps to 44; `0xFFFFFF10` ≥ 44 → **elapsed ✘** (284 ms too early)
356 ms after start (after the wrap)	`0x00000064` (100)	256 + 100 = 356 → 356 ≥ 300: **elapsed ✔**	—

The subtraction always measures the real distance; the addition breaks as soon as `start + delay` passes `0xFFFFFFFF`.

5.5 Idle Loop with `wfi`

When there is nothing to do, the CPU should sleep instead of spinning. The `wfi` instruction puts it to **sleep until the next interrupt.**

```
uint32_t last = systick_uptime_ms();
for ( ;; )
{
    __asm volatile ( "wfi" );           /* sleep until an interrupt */
    uint32_t now = systick_uptime_ms();
    if ( now - last >= 1000 )           /* safe across wraparound */
    {
        last += 1000;
        kprintf( "uptime: %u ms\n", now );
    }
}
```

Element	Effect
`wfi`	sleeps until an exception is pending; the loop runs once per interrupt, at least every 10 ms
`last += 1000`	schedules each print 1 s after the previous one: no drift (unlike `last = now`)

5.6 Inspecting the Exception Frame in gdb

The stacked frame described in the exception model of the Cortex-M3 chapter can be observed directly with gdb, by stopping at the first instruction of `SysTick_Handler`.

```
(gdb) break *SysTick_Handler        # stop at the very first instruction
(gdb) continue
(gdb) x/8wx $sp                     # r0 r1 r2 r3 / r12 lr pc xPSR
```

```
0x203fffd0:  0x00000000  0x00000000  0x00000007  0x00000000
0x203fffe0:  0x00000000  0x0000050d  0x00000510  0x01000000
(gdb) p/x $lr
$1 = 0xfffffff9
(gdb) info symbol 0x510
main + 28 in section .text
```

Observation	Meaning
`break *SysTick_Handler`	stops before the prologue, while `sp` points at the frame
Stacked `PC` = `0x510`	instruction after `wfi` in `main`
Stacked `r2` = 7	value just written to `CTRL`
`lr` = `0xFFFFFFF9`	return to Thread mode, MSP

6 The NVIC and Interrupt-Driven UART Transmission

Device interrupts are managed by the **NVIC** (Nested Vectored Interrupt Controller). Using it, the UART can transmit in the background: the program puts bytes in a buffer, and an interrupt handler sends them one by one.

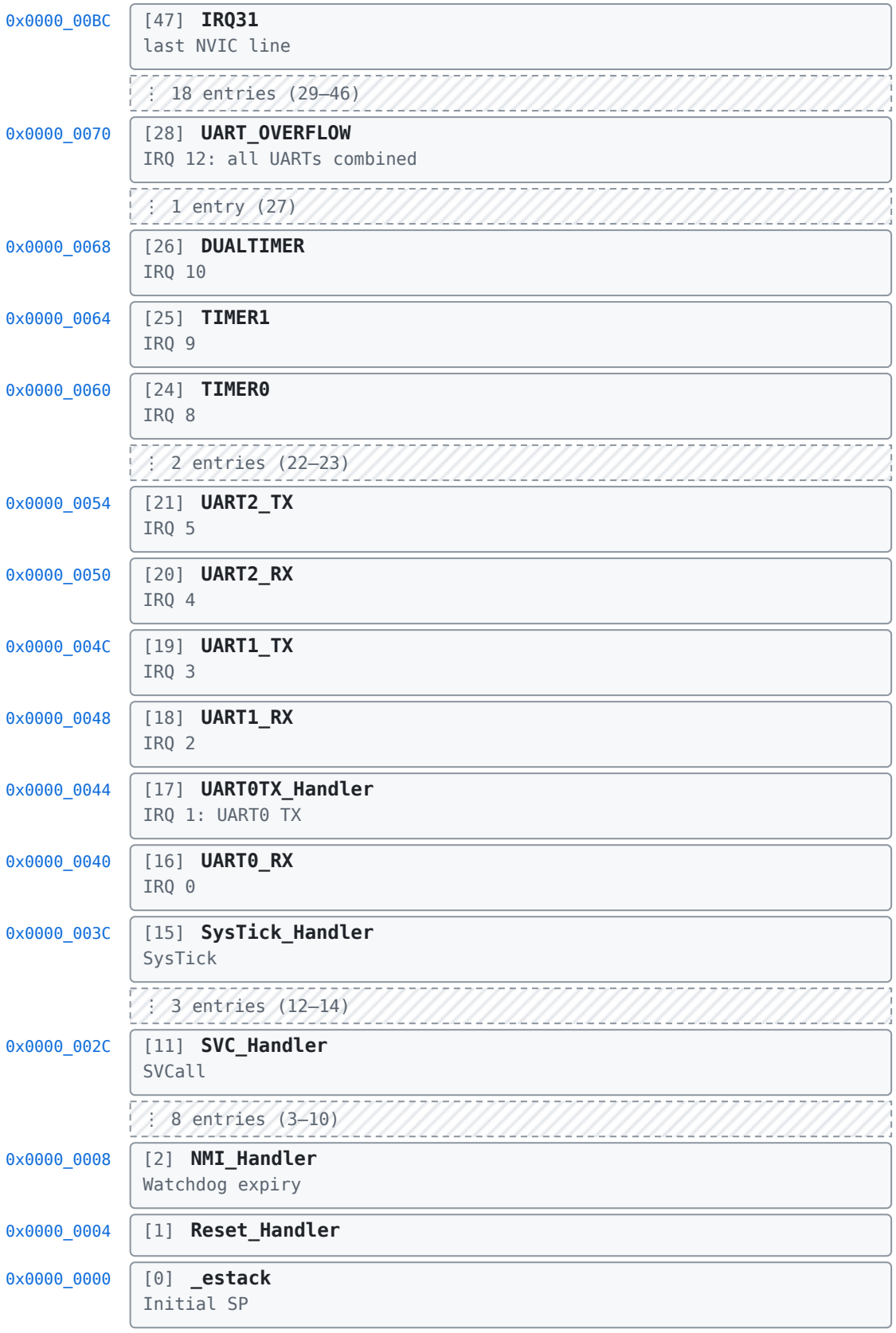
6.1 IRQ Lines and Handlers on the MPS2-AN385

Device interrupts reach the CPU through the NVIC, whose registers are described in the Cortex-M3 chapter. The `mps2-an385` board connects 32 interrupt request lines, **IRQ 0–31**, to it. Every IRQ is also an exception with its own vector table slot: IRQ n is exception 16 + n, and its handler address is stored at offset 4 × (16 + n).

> **UART0 TX**
>
> IRQ 1 → exception 16 + 1 = 17 → slot address 4 × 17 = 68 = `0x44`, which holds the address of `UART0TX_Handler`.

IRQ	Source	IRQ	Source
0	UART0 RX	8	TIMER0
1	UART0 TX	9	TIMER1
2 / 3	UART1 RX / TX	10	dual timer
4 / 5	UART2 RX / TX	12	UART overflow (all UARTs; never fires in QEMU)

Figure 4.4 — Vector table with device interrupts (48 words, 0xC0 bytes)



The vector table grows from 16 to 48 words. As for the core exceptions, every device handler name is a `.weak` alias of `Default_Handler`, and a C function with the same name (for example `UART0TX_Handler`) replaces it at link time without editing the table.

The board's watchdog is connected to the **NMI** (exception 2) instead of an ordinary IRQ line. A watchdog is a timer that the software must reload periodically; if the program hangs and stops reloading it, the watchdog fires so that the system can recover. It uses the NMI because a hung program may have masked all ordinary interrupts: `PRIMASK` does not block the NMI, and its fixed priority is above every configurable one.

6.2 Enabling, Acknowledging, and Prioritising the UART Interrupt

```
NVIC->IPR[UART0TX_IRQn] = 0xC0;                              /* priority of IRQ 1 */
NVIC->ISER[UART0TX_IRQn / 32] = 1UL << ( UART0TX_IRQn % 32 ); /* enable IRQ 1 */
```

The first line writes the priority byte of IRQ 1; the second sets its bit in the enable register. Set and clear registers ignore 0 bits, so a plain assignment is enough and no `|=` is needed. The UART TX handler then runs only if three conditions hold at the same time:

1. the **device** may raise the interrupt (UART TX: `CTRL.TXIRQEN`, bit 2);
2. the **NVIC** line is enabled in `ISER`;
3. its priority is more urgent than that of the running code, and `PRIMASK` = 0.

NVIC lines are **level-sensitive**: a handler that does not clear the device flag is re-entered forever. The UART handler therefore starts by **acknowledging at the device**, with the write-1-to-clear store `CMSDK_UART0->INTSTATUS = UART_INT_TX;`.

Priorities decide which exception runs first and which can interrupt another. A lower value means more urgent, and only a **strictly more urgent** exception preempts the running one; the others stay **pending**. The kernel uses the following policy:

Exception	Priority
Faults	`0x00` or more urgent
SVCall	`0x80`
UART0 TX (IRQ 1)	`0xC0`
SysTick	`0xE0`

6.3 Races on Shared Data and Critical Sections

When `main` and a handler share data, an interrupt arriving in the middle of an update can corrupt it. A critical section prevents this by temporarily masking interrupts.

`volatile` forces real memory accesses but does not make an update indivisible:

```
/* count++ */
    ldr  r1, [r0]       /* r1 = count            (say 5)                    */
    adds r1, r1, #1     /* r1 = 6   <- the handler runs here, sets count = 6 */
    str  r1, [r0]       /* count = 6: the handler's update is lost          */
```

Operation on shared data	Safe?
Single aligned 32-bit read or write	yes
Read-modify-write (`++`, `+=`, `&=`, OR-assignment)	no: **lost update** (a **race**)
Update of several related variables	no

A **critical section** masks interrupts through `PRIMASK`, which C cannot name; two assembly functions provide nestable masking:

```
    .syntax unified
    .cpu cortex-m3
    .thumb
    .section .text,"ax",%progbits

/* uint32_t irq_save(void) */
    .global irq_save
    .type irq_save, %function
    .thumb_func
irq_save:
    mrs r0, primask             /* old = PRIMASK; r0 = return value */
    cpsid i                     /* PRIMASK = 1: interrupts masked */
    bx lr                       /* return old (in r0) */
    .size irq_save, . - irq_save

/* void irq_restore(uint32_t old) */
    .global irq_restore
    .type irq_restore, %function
    .thumb_func
irq_restore:
    msr primask, r0             /* PRIMASK = old (first argument, r0) */
    bx lr                       /* return */
    .size irq_restore, . - irq_restore
```

```
uint32_t a = irq_save();     /* PRIMASK 0 -> 1, a = 0 */
uint32_t b = irq_save();     /* PRIMASK already 1, b = 1 */
irq_restore( b );            /* PRIMASK stays 1: a plain cpsie here would unmask too early */
irq_restore( a );            /* PRIMASK back to 0 */
```

Rule	Reason
Restore the saved value, never `cpsie i`	sections **nest** correctly
Only the main code needs it	a handler cannot be interrupted by the main code
Short, never waiting	interrupts stay pending until `irq_restore`

6.4 Ring Buffer

A ring buffer is the data structure that connects the program (which produces bytes) and the interrupt handler (which consumes them).

A **ring buffer** is an array used as a FIFO: producer at `head`, consumer at `tail`, both wrapping.

“Pasted image 20261018202856.png” could not be found.

Operation	Expression
Next index	`( i + 1 ) & ( SIZE - 1 )`, equal to `% SIZE` for a power of two, with no division
Empty	`head == tail`
Full	`( ( head + 1 ) & ( SIZE - 1 ) ) == tail`

Each index has one writer. One slot stays unused, so 64 entries hold **63** bytes: otherwise full and empty would both satisfy `head == tail`.

6.5 Interrupt-Driven Transmitter

The interrupt-driven UART driver combines the previous pieces: the ring buffer, the critical section, and the TX interrupt handler.

```
#define TX_BUF_SIZE 64                  /* a power of two */

static char tx_buf[TX_BUF_SIZE];        /* bytes waiting to be sent */
static volatile unsigned tx_head;       /* next free slot: written by uart_write */
static volatile unsigned tx_tail;       /* oldest byte: written by the handler (and priming) */
static volatile int tx_busy;            /* 1 while the UART sends and its IRQ will come */

static int tx_empty( void ) { return tx_head == tx_tail; }
static int tx_full( void )  { return ( ( tx_head + 1 ) & ( TX_BUF_SIZE - 1 ) ) == tx_tail; }

static char tx_pop( void )              /* take the oldest byte; buffer not empty */
{
    char c = tx_buf[tx_tail];
    tx_tail = ( tx_tail + 1 ) & ( TX_BUF_SIZE - 1 );
    return c;
}

void uart_init( void )
{
    CMSDK_UART0->BAUDDIV = 16;
    CMSDK_UART0->CTRL = UART_CTRL_TXEN | UART_CTRL_RXEN;
```

```
    tx_head = 0;
    tx_tail = 0;
    tx_busy = 0;
    NVIC->IPR[UART0TX_IRQn] = 0xC0;                              /* below SVCall, above
SysTick */
    NVIC->ISER[UART0TX_IRQn / 32] = 1UL << ( UART0TX_IRQn % 32 ); /* enable IRQ 1 */
}

unsigned uart_write( const char *buf, unsigned len )
{
    unsigned accepted = 0;
    uint32_t saved = irq_save();               /* the handler must not run during the updates
*/

    while ( accepted < len && !tx_full() )      /* copy what fits */
    {
        tx_buf[tx_head] = buf[accepted];
        tx_head = ( tx_head + 1 ) & ( TX_BUF_SIZE - 1 );
        accepted++;
    }

    if ( !tx_busy && !tx_empty() )              /* idle: no IRQ will come, so prime it */
    {
        CMSDK_UART0->CTRL |= UART_CTRL_TXIRQEN; /* FIRST enable the TX interrupt ... */
        while ( CMSDK_UART0->STATE & UART_STATE_TXBF )
        {
        }                                       /* (a polled byte may still be leaving) */
        CMSDK_UART0->DATA = (uint32_t) (unsigned char) tx_pop(); /* ... THEN write */
        tx_busy = 1;
    }

    irq_restore( saved );
    return accepted;
}

void UART0TX_Handler( void )
{
    CMSDK_UART0->INTSTATUS = UART_INT_TX;       /* acknowledge at the device (W1C) */

    if ( !tx_empty() )
    {
        CMSDK_UART0->DATA = (uint32_t) (unsigned char) tx_pop(); /* send the next byte */
    }
```

```
    else
    {
        CMSDK_UART0->CTRL &= ~UART_CTRL_TXIRQEN;  /* nothing left: interrupt off */
        tx_busy = 0;                              /* transmitter idle */
    }
}
```

Component	Role
`uart_write` (**producer**)	critical section; copies what fits and **returns the count accepted**, never waits; **primes** an idle transmitter with the first byte
`UART0TX_Handler` (**consumer**)	acknowledges, sends the next byte or disables `TXIRQEN`; no critical section needed

PRIMING ORDER

The TX interrupt fires when a byte has been sent **and** `TXIRQEN` was set when that byte was written. Enabling it while idle raises nothing; enabling it after the write misses that byte. Correct order: first `TXIRQEN`, then the first byte.

160 BYTES THROUGH THE 64-BYTE BUFFER

`uart_write` is called 3 times (63, 63, 34 accepted); `UART0TX_Handler` runs **160 times**, the last one disabling `TXIRQEN`.

7 System Calls and the Kernel Boundary

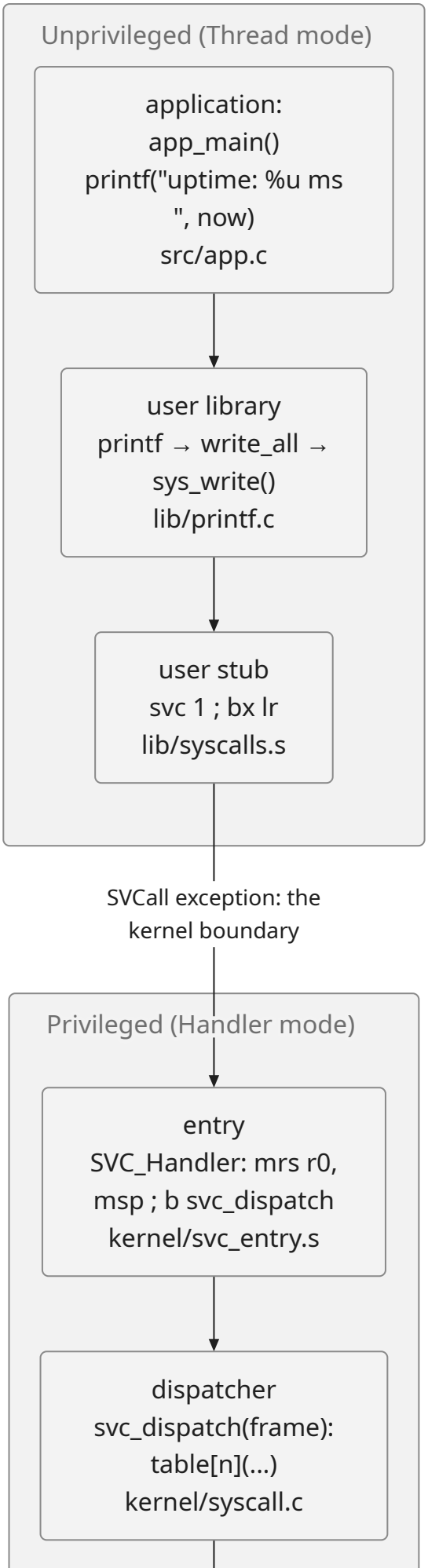
Application code can be made unprivileged, so that it cannot reconfigure the machine. It then needs a controlled way to ask the kernel for services: the system call, implemented with the `svc` instruction.

7.1 Privilege Model and Layered Structure

The Cortex-M distinguishes **privileged** and **unprivileged** execution. The system is organized in layers, with the application on the unprivileged side and the kernel and drivers on the privileged side.

Mechanism	Behaviour
`CONTROL.nPRIV` = 1 in Thread mode	execution becomes unprivileged; one-way (unprivileged code cannot clear the bit)
Unprivileged access to the System Control Space (SysTick, NVIC, SCB)	**BusFault**
`svc #n`	allowed; raises **SVCall**, whose handler runs in Handler mode, always privileged
Return from SVCall	resumes after `svc`, still unprivileged

Figure 4.5 — Layers on the two sides of the kernel boundary



Without an MPU the UART stays reachable from unprivileged code, so "all I/O through system calls" is a **design choice**, with one hard reason: `cpsid i` **is silently ignored unprivileged**, so `uart_write` is correct only when privileged.

The mechanism is the same as in a desktop operating system:

Aspect	Linux on a desktop	Bare-metal Cortex-M3
Restricted code	user-mode process	application in unprivileged Thread mode
Trap instruction	`syscall` (x86-64), `svc` (ARM)	`svc #n`
Privileged side	the kernel	`SVC_Handler` and the dispatcher
Service	`write()` puts bytes on the terminal	`sys_write()` sends bytes to the UART driver

The restricted side cannot do the job itself, so it traps into code that can, and that code decides what to do. This is the core of every operating system's protection model.

7.2 The System-Call ABI

Caller and kernel must agree on how a request is expressed at the machine level: which service is requested, where the arguments are, and where the result goes.

The **ABI** (Application Binary Interface): service number in the immediate of `svc #n`, arguments in `r0`–`r2`, result in `r0`.

Number	Service	Arguments	Result
1	`sys_write`	`buf`, `len`	bytes accepted, 0 to `len` (`uart_write`)
2	`sys_read`	none	one received byte, or `-1` (`uart_getc_polled`)
3	`sys_uptime`	none	milliseconds since start (`systick_uptime_ms`)
0, ≥ 4	none	none	`-1`

No service ever waits: the caller handles partial results.

7.3 Encoding of `svc #n` and Decoding from the Stacked Frame

The service number is not passed in a register but encoded inside the `svc` instruction itself; the kernel reads it, together with the arguments, from the stacked frame.

```
$ arm-none-eabi-objdump -d build/scratch-os.axf          one instruction per line
000008e0 <sys_write>:
 8e0:   df01    svc  1
 8e2:   4770    bx   lr

$ arm-none-eabi-objdump -s -j .text build/scratch-os.axf  raw bytes, memory order
 08e0 01df7047 02df7047 03df7047
```

svc_1 16-bit **= 0xDF01**

Thumb encoding of svc #1 (one halfword)



Bits	Field	Value	Description
15:8	`OPCODE`	0xDF (223)	0xDF identifies svc
7:0	`IMM8`	0x1 (1)	Service number n

Memory is **little-endian**: `objdump -d` shows the halfword `df01`, `objdump -s` the bytes `01 df`. With `uint32_t *frame` pointing at the stacked frame (see the exception model in the Cortex-M3 chapter; `frame[6]` = `pc`):

Item	Location	Reason
Service number `n`	`( ( uint8_t * ) frame[6] )[-2]`	the stacked `pc` points **after** the 2-byte `svc`
Arguments	`frame[0]` – `frame[2]`	stacked by the hardware: no copy needed
Result	**`frame[0]`**	the exception return pops it into the caller's `r0`

7.4 Assembly Entry Point `SVC_Handler`

To read the stacked frame, the kernel must know where it is at the very first instruction of the handler; this requires a short entry point in assembly.

The kernel needs **MSP** at entry to find the frame, but a C prologue moves `sp` first:

```
0000027a <svc_dispatch>:
 27a:  b580   push  {r7, lr}     @ sp = sp - 8
 27c:  b084   sub   sp, #16      @ sp = sp - 16
 27e:  af00   add   r7, sp, #0
```

`sp` has moved by a compiler-dependent 24 bytes. A handler that must find the frame needs an assembly entry:

```
    .global SVC_Handler         /* replaces the weak alias in the vector table */
    .type SVC_Handler, %function
    .thumb_func
SVC_Handler:
    mrs r0, msp                 /* r0 = MSP: frame = address of the 8 saved registers */
    b svc_dispatch              /* svc_dispatch(frame); lr still = EXC_RETURN */
    .size SVC_Handler, . - SVC_Handler
```

Instruction	Purpose
`mrs r0, msp`	frame address in `r0`, the first AAPCS argument of `svc_dispatch(uint32_t *frame)`

Instruction	Purpose
`b` (not `bl`)	**tail branch**: `bl` would overwrite `lr` and lose `EXC_RETURN`; with `b`, the normal return of `svc_dispatch` through `lr` performs the exception return directly

7.5 Dispatch Table and User-Side Stubs

On the kernel side, a table maps each service number to its implementation; on the application side, tiny assembly stubs turn a C call into an `svc`.

```
/* A service: up to 3 arguments (caller's r0-r2), one result (caller's r0). */
typedef uint32_t ( *syscall_fn )( uint32_t a0, uint32_t a1, uint32_t a2 );

static uint32_t sys_write_impl( uint32_t a0, uint32_t a1, uint32_t a2 )
{
    (void) a2;
    return uart_write( ( const char * ) a0, a1 );   /* bytes accepted, 0..len */
}

static uint32_t sys_read_impl( uint32_t a0, uint32_t a1, uint32_t a2 )
{
    (void) a0; (void) a1; (void) a2;
    return ( uint32_t ) uart_getc_polled();         /* byte, or -1 */
}

static uint32_t sys_uptime_impl( uint32_t a0, uint32_t a1, uint32_t a2 )
{
    (void) a0; (void) a1; (void) a2;
    return systick_uptime_ms();
}

static const syscall_fn table[] = {             /* entry 0 stays empty (0) */
    [SYS_WRITE]  = sys_write_impl,
    [SYS_READ]   = sys_read_impl,
    [SYS_UPTIME] = sys_uptime_impl,
};

#define TABLE_SIZE ( sizeof( table ) / sizeof( table[0] ) )

void svc_dispatch( uint32_t *frame )
{
    uint8_t n = ( ( uint8_t * ) frame[6] )[-2];  /* low byte of svc #n before the stacked pc
*/
```

```
    if ( n >= TABLE_SIZE || table[n] == 0 )
    {
        frame[0] = ( uint32_t ) -1;             /* unknown service */
        return;
    }
    frame[0] = table[n]( frame[0], frame[1], frame[2] );  /* result into the stacked r0 */
}
```

Element	Meaning
`syscall_fn`	one signature for all services, so they fit one array
Designated initializers	each entry at its number; entry 0 stays empty; `TABLE_SIZE` follows automatically
Bounds and null check	unknown service answers `-1`

```
/* int sys_write(const char *buf, unsigned len): buf in r0, len in r1 */
    .global sys_write
    .type sys_write, %function
    .thumb_func
sys_write:
    svc #1                      /* svc(SYS_WRITE, buf, len) */
    bx lr                       /* return r0 (set by the kernel) */
    .size sys_write, . - sys_write
```

`sys_read` (`svc #2`) and `sys_uptime` (`svc #3`) are identical. The number is part of the encoding, so it is a literal kept in sync by hand with the C header. In gdb, `x/2bx ((unsigned *)$sp)[6] - 2` at `SVC_Handler` shows `0x03 0xdf` (`svc #3`), and `finish` from `svc_dispatch` lands on the stub's `bx lr` : the return was the exception return.

7.6 User-Space `printf` , `write_all` , and `sleep_ms`

Unprivileged code cannot use `kprintf` , so the application gets its own `printf` , built only on system calls.

Aspect	`kprintf` (kernel)	`printf` (user)
Each character goes to	`uart_putc_polled`	a 64-byte buffer on the stack
When the buffer is full, or at the end	—	`write_all(buf, used)`
Path to the UART	direct register access (privileged)	`sys_write` → `svc #1`

```
#define PRINTF_BUF_SIZE 64      /* characters per sys_write */

void write_all( const char *buf, unsigned len )
{
    while ( len > 0 )
```

```
    {
        int n = sys_write( buf, len );  /* the kernel takes what fits */
        buf += n;
        len -= n;
        if ( n == 0 )
        {
            __asm volatile ( "wfi" );   /* full: sleep until an interrupt */
        }
    }
}

struct printf_ctx
{
    char buf[PRINTF_BUF_SIZE];  /* characters not sent yet */
    unsigned used;              /* how many of buf are in use */
    int total;                  /* characters produced so far */
};

static void printf_out( char c, void *ctx )    /* store one char, flush when full */
{
    struct printf_ctx *p = ctx;
    p->buf[p->used] = c;
    p->used++;
    p->total++;
    if ( p->used == PRINTF_BUF_SIZE )
    {
        write_all( p->buf, p->used );
        p->used = 0;
    }
}

int printf( const char *fmt, ... )
{
    struct printf_ctx p;                    /* on the stack, about 72 bytes */
    p.used = 0;
    p.total = 0;
    va_list ap;
    va_start( ap, fmt );
    vformat( printf_out, &p, fmt, ap );
    va_end( ap );
    write_all( p.buf, p.used );             /* send what is left */
    return p.total;
}
```

```
void sleep_ms( uint32_t ms )
{
    uint32_t start = sys_uptime();
    while ( sys_uptime() - start < ms )     /* safe across wraparound */
    {
        __asm volatile ( "wfi" );
    }
}
```

Element	Design point
64-byte buffer	one system call per 64 characters
`write_all`	handles **partial writes**; on 0 accepted, `wfi` (allowed unprivileged) until the TX interrupt frees room
`sleep_ms`	the **waiting happens in Thread mode**, where interrupts run

> ⚠ **HIDDEN `MEMSET` IN A FREESTANDING BUILD**
>
> `struct printf_ctx p = { .used = 0, .total = 0 };` fails to link (`undefined reference to 'memset'`): GCC zero-fills the array by calling `memset` , absent without a C library.

7.7 Kernel Start and Unprivileged Application

`main` initializes the system while still privileged, then drops privilege and hands control to the application, which from then on uses only system calls.

```
int main( void )
{
    uart_init();                                /* 1. console first */
    kprintf( "scratch-os: kernel started\n" );  /* 2. polled print */
    systick_init();                             /* 3. 10 ms tick */
    kprintf( "dropping to unprivileged mode\n" );
    __asm volatile (            /* 4. CONTROL = CONTROL | 1; (nPRIV = 1) */
        "mrs r0, control    \n" /* r0 = CONTROL */
        "orr r0, r0, #1     \n" /* r0 |= 1: set nPRIV */
        "msr control, r0    \n" /* CONTROL = r0: now unprivileged */
        "isb                \n" /* refetch under the new rules */
        : : : "r0", "memory" );
    app_main();                                 /* 5. never returns */
    for ( ;; ) { }                              /* not reached */
}

void app_main( void )
```

```
{
    printf( "app: hello from unprivileged code\n" );
    uint32_t last = 0;                          /* when the uptime was last printed */

    for ( ;; )
    {
        uint32_t now = sys_uptime();
        if ( now - last >= 1000 )               /* one second passed? */
        {
            printf( "uptime: %u ms\n", now );
            last += 1000;
        }
        int c = sys_read();                     /* never waits: -1 = no key */
        if ( c != -1 )
        {
            printf( "key: %c\n", c );
        }
        sleep_ms( 10 );
    }
}
```

Element	Reason
Order: console, tick, drop	printing first; the tick serves `sys_uptime` and `sleep_ms`
`isb`	discards instructions fetched under privileged rules
Clock comparison	counting iterations would drift

```
scratch-os: kernel started
dropping to unprivileged mode
app: hello from unprivileged code
uptime: 1000 ms
key: a
key: b
uptime: 2000 ms
```

The first two lines come from polled `kprintf`, the rest through `printf` → `sys_write` → `svc #1` → `uart_write` → `UART0TX_Handler`. In gdb, `$control` is `0x1` even inside `sys_write_impl` (privileged only because in Handler mode), and `$xpsr & 0x1ff` = 11 (SVCall).

7.8 Kernel Rules: No `svc` Inside the Kernel, Never Wait

Two rules keep the kernel correct; breaking either one stops the system, as the following failures show.

No system calls in kernel code. An `svc` is **synchronous** and cannot stay pending: if not more urgent than the

running code, it **escalates** to HardFault (HFSR bit 30, **FORCED**, e.g. `HFSR=0x40000000`).

Where `svc` executes	Current priority	Result
`app_main`, `main` (Thread mode)	none (lowest)	taken
`SysTick_Handler` / `UART0TX_Handler`	`0xE0` / `0xC0`	taken: SVCall is more urgent and nests
`SVC_Handler` and anything it calls	`0x80`	HardFault
A fault handler	`0x00` or more urgent	HardFault
Between `irq_save` and `irq_restore`	masked (`PRIMASK` = 1)	HardFault

The kernel never waits. If `sys_write_impl` loops until all bytes are accepted, a 130-byte write prints one byte and freezes: only `UART0TX_Handler` (`0xC0`) can drain the buffer, but the CPU is in SVCall (`0x80`), more urgent, so TX and SysTick (`0xE0`) stay pending forever. Hence `sys_write` returns the accepted count and `write_all` waits in **Thread mode**.

> **TWO KERNEL INVARIANTS**
>
> Never block inside a system call (partial results, waiting in user space). Never issue a system call from the kernel (output via polled `kprintf`).